Software Engineering

Software Processes - Part 1

Okba Tibermacine
National School of Artificial Intelligence

October 7, 2025

Outline

- Revision for the Previous Week
- Software Process and Activities
- Software Process Models - Part 1
- Traditional Methodologies

From last week

Definition of Software Engineering

A discipline that integrates processes, methods and tools for the analysis, design, development, testing, and maintenance of software systems.

The term was coined at a conference in 1968.

From last week

The goal of software development is to develop:

High quality software that meets customers' real needs

From last week

Davis's principles (1994):

- Make quality Number 1
- High Quality Software is possible
- Give products to customer early
- Determine the problem before writing requirements
- Evaluate design alternatives
- Use an appropriate process model
- People are the key to success
- Use different languages for different phases
- Minimize intellectual distance
- Put technique before tools
- Get it right before you make it faster
- Inspect code
- Good management is more important than good technology

From last week

Qualities of Industrial-strength software Product:

- Correctness (Functionality)
- Usability and Acceptability
- Reliability
- Robustness
- Performance and Efficiency
- Security
- Portability
- Maintainability

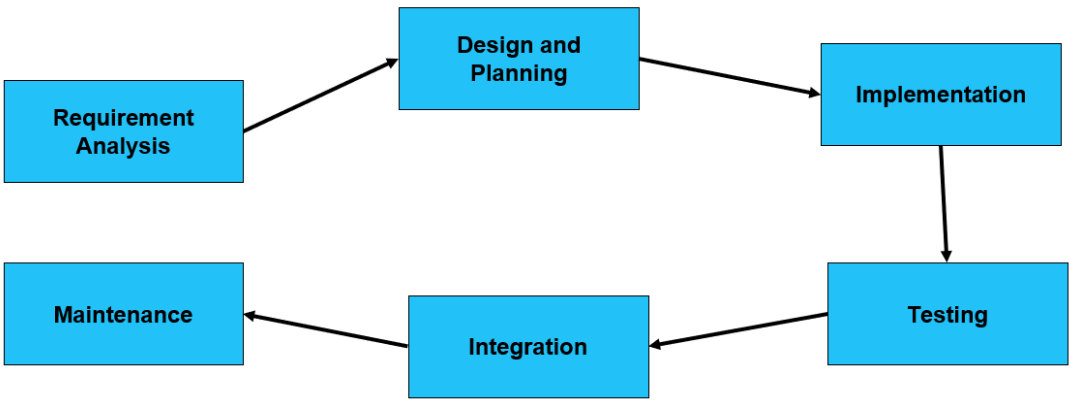
from last week

To develop a Software:

- **Requirement Analysis:** Understand the problem
- **Design:** Plan the solution
- **Implementation:** Code the solution
- **Testing:** Verify the solution
- **Deployment:** Release the solution
- **Maintenance:** Update the solution

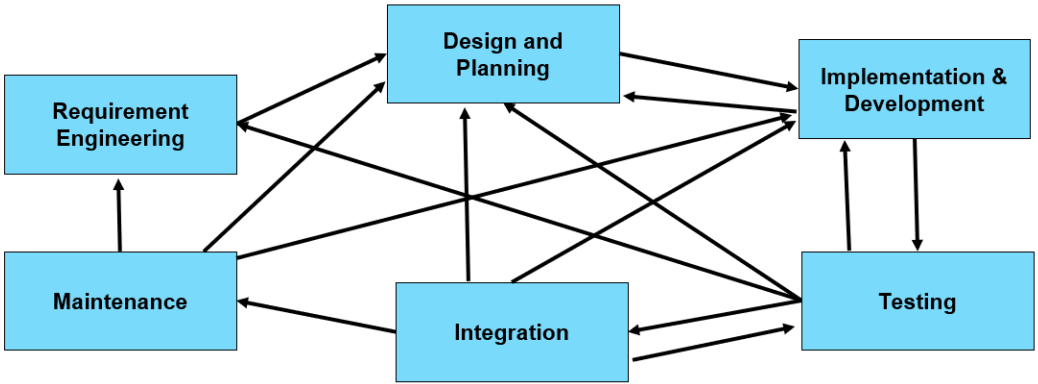
From last week

Ideal software process



From last week

But in reality :



From Last week

Plan-Driven Processes

- All activities are planned in advance
- Progress measured against the plan
- Emphasis on documentation
- Requirements defined upfront
- Sequential phases
- Example: Waterfall Model

Agile Processes

- Planning is incremental
- Easier to change process
- Less documentation
- Requirements evolve
- Iterative development
- Example: Scrum, XP

Key Difference: Plan-driven processes work best when requirements are stable and well-understood, while Agile processes are better suited for projects with changing requirements.

Software Processes and Activities

What is a Software Process?

Software Process

A structured set of activities required to develop a software system. It is a collection of activities, actions, and tasks performed during the creation of a software product, along with the logical order and relationships among them

Function of a Software Process

A software process defines *who* does *what*, *when*, and *how* to transform requirements into a working software system.

Processes Components

The process encompasses **four interconnected elements**:

1. **Activities** : The tasks, actions, and operations performed during development.
2. **Roles** : Reflects the responsibilities of the people involved in the activity.
3. **Artifacts/deliverables** : The tangible outcomes of a process activity (e.g., Specification Documentation, Architecture Design,...)
4. **Workflows** : The sequence and dependencies of activities.

Process Activity

Definition

An **Activity** is a unit of work performed within a software process. It is **performed by a Role**, produces or modifies **Artifacts**, and is subject to **Pre- and Post-Conditions**.

Process Activity

Definition

An **Activity** is a unit of work performed within a software process. It is **performed by a Role**, produces or modifies **Artifacts**, and is subject to **Pre- and Post-Conditions**.

Pre- and Post-Conditions

Pre- and Post-Conditions are statements that are true before and after a process activity has been enacted or a product produced.

Process Activity

Definition

An **Activity** is a unit of work performed within a software process. It is **performed by a Role**, produces or modifies **Artifacts**, and is subject to **Pre- and Post-Conditions**.

Pre- and Post-Conditions

Pre- and Post-Conditions are statements that are true before and after a process activity has been enacted or a product produced.

Example: Design Activity

- **Role:** Software Architect
- **Pre-condition:** Requirements Specification must be completed.
- **Post-condition:** Architectural Design must be reviewed by an expert.

Process Activity

Process Activities: The following information needs to be defined:

- **Deliverable:** The outcome of the activity
- **Duration:** How long the activity should take
- **Resources:** What resources are needed
- **Dependencies:** What other activities depend on this one

Generic activities

Generic Activities in a Software Engineering Process:

- Requirements Specification
- Design and Planning
- Coding/Implementation/Construction
- Testing
- Integration
- Maintenance

Umbrella Activities

SE process framework activities are complemented by a number of umbrella activities that provide support throughout a software project:

Examples of Umbrella activites

- Software Project Tracking and Control : *monitors project progress and performance.*
- Risk Management :*identifies, analyzes, and mitigates potential project risks*
- Software Quality Assurance: *ensures that quality standards and procedures are followed.*
- Technical Reviews : *assess work products for defects and compliance with requirements.*
- Measurement and Metrics : *collect and analyze data to improve processes and products.*

Tasks

A Task is a work unit with a specific purpose related to the larger goal defined within an activity.
Tasks can be of different types :

Examples of tasks

- Implementation of new Functionality
- Fix Bug of the system A
- Buy some equipment
- Negotiate a deal with a supplier
- Send Invoices to Customer
- Collect data from end-users
- Hire a Translator Git Repository ...

Tasks

- Task is usually assigned to a single employee at a time
- Task can be reassigned to another employee
- Tasks are assigned to employees based on their skills and availability
- A task that was initially assigned to an employee can be unassigned

Task Status

Possible status

- **Backlog** → New ideas, new features, possible bugs, or any other tasks to do.
- **In Plan** → Features, tasks, or bugs that have been confirmed to be taken into consideration.
- **Started** → Tasks that employees have started working on.
- **Completed** → Tasks that employees claim to have completed, pending review by their manager.
- **To Deliver** → Task, feature, or bug fix pending integration or release into the production system.
- **To Accept** → Task, feature, or bug pending acceptance by the client or task owner.
- **Closed** → Task fully completed, delivered, and accepted.
- **Trash** → Task, feature, or functionality that is ignored or abandoned.

Task workflow

Example of possible task workflow : To do → In Progress → Done

Idea/Backlog →To do → In progress → To Review →Completed → Delivered

Backlog →In Plan → Started →Completed → To Deliver → To Accept →Closed → Trash

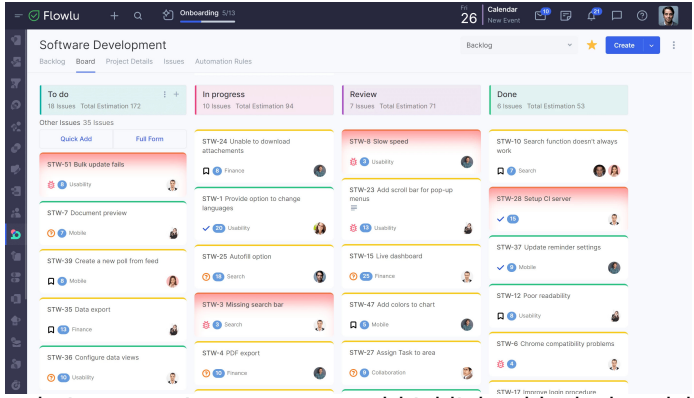
Note

Different organizations customize their workflow depending on their **development process**, **tooling**, and **team size**.

Tools to track task statuses

Many project management tools help track and visualize task status throughout the software development process.

- **Jira**
- **Trello**
- **Asana**
- **...**



This visual approach provides clarity on project progress and highlights blocked or delayed tasks.

Tasks

Policies for completing a Task:

- A task should never be marked completed by the starter
- A task should be marked completed by the reviewer
- A task should be marked completed only after it is reviewed

Tasks

The complexity of tasks can differ and if necessary:

- Sub-tasks can be created inside a task
- Sub-tasks can be assigned to different employees
- Sub-tasks can have their own workflow

Task

Due date: the project manager or the executor of the task must set a due date indicating when the task should be completed.

Software Process Models

Software Process Models

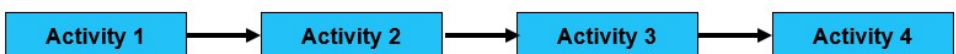
What's a Process Model:

- is a description of what activities/tasks need to be performed in what sequence and when during a software development project.
- is an abstract representation of how a process is conducted in terms of structuring and ordering the tasks to develop the software product
- i.e. The model describes the process flow

Software Process Models

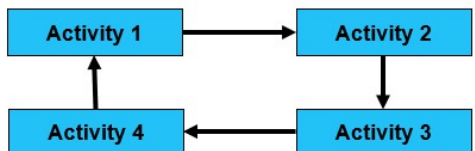
What's a Process Model:

It can be linear workflow:



Software Process Models

What's a Process Model:

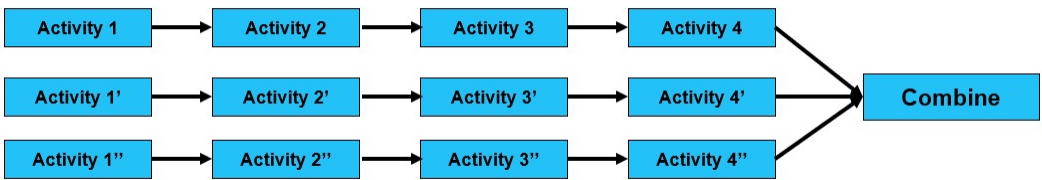


Or iterative/cyclic workflow: Activities connected in cycles with feedback loops

Software Process Models

What's a Process Model:

Or even parallel workflows:



Multiple activities happening simultaneously and then combining

Software Process Models

- Why a software process model :
  - is to provide guidance for systematically **coordinating** and **controlling** the tasks that must be performed in order to achieve the end product and the project objective (= **Be organized**)
  - What if the development is done by **one person ?** do we still need a adopt a process model ?

Software Process Models

- Why a software process model :
  - is to provide guidance for systematically **coordinating** and **controlling** the tasks that must be performed in order to achieve the end product and the project objective (= **Be organized**)
  - What if the development is done by **one person ?** do we still need a adopt a process model ?

There is a disagreement about the need to use a specific process model for a one-man team but :
It is always recommended to adopt a process model for keeping yourself organized and keep the growing list of tasks/features/bugs under control.

Software Process Models

Why a software process model:

- Software product needs to be of high quality
- The quality and product attributes are largely **determined** by the process used to develop it.

Plan-driven processes

Types of Software Processes

Plan-Driven Processes:

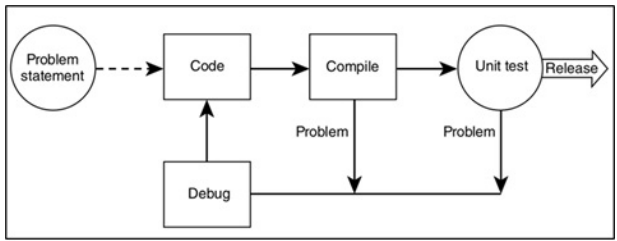
are processes where all of the process activities are planned in advance and progress is measured against this plan.

Agile Processes:

Planning is incremental and it is easier to change the process to reflect changing customer requirements.

Software Process Models: Simple Process Models

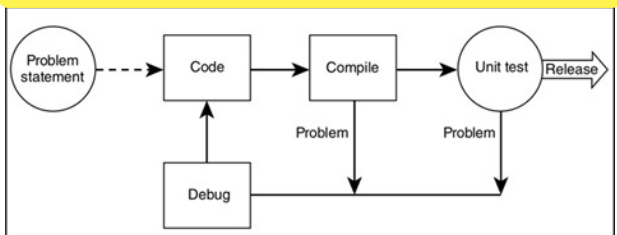
- **Code-and-Fix Model** :
  - No specification or design are made, developers would aim to focus heavily on the coding aspect
  - Anytime, there is an error or a functionality to be implemented, the developers will intervene the fix the error or do further coding.
  - It is rare that the documentation are produced



Software Process Models: Simple Process Models

- **Code-and-Fix Model** :
  - No specification or design are made, developers would aim to focus heavily on the coding aspect
  - Anytime, t… e developers will intervene t…
  - It is rare tl…

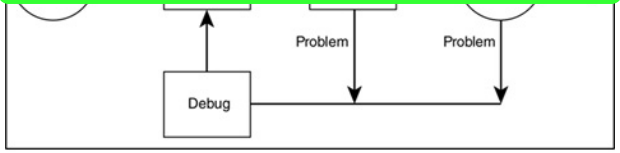
Who does the testing ?



Software Process Models: Simple Process Models

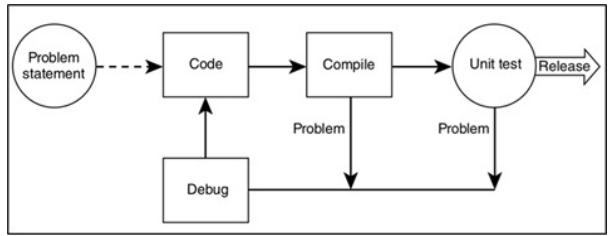
- **Code-and-Fix Model** :
  - No specifi… s heavily on the coding asp…
  - Anytime, t… e developers will intervene t…
  - It is rare tl…

The developers themselves do the testing because : There is no documentation or specification to tell the testers about the software, its functionalities, problem to be solved...



Software Process Models: Simple Process Models

- **Code-and-Fix Model** :
  - As
    - software products are becoming more complex
    - Increasing number of tasks with changing requirements
    - More people are involved
  - This model would not be a fit to most large software projects
    - Software is not maintainable
    - Higher costs
    - Quality is questionable



Software Process Models: Simple Process Models

- **No Code Process Model**:
  - Aims to develop software products using "*Drag and Drop*" Platforms offering pre-built components to conduct various types of business logic.
  - Existing Tools/Platforms :
    - *Bubble*
    - *Visual Programming Languages*
  - With or Against ?
    - *Scalability ?*
    - *Custom Business Logic ?*
    - *Sovereignty ?*

Software Process Models: Simple Process Models

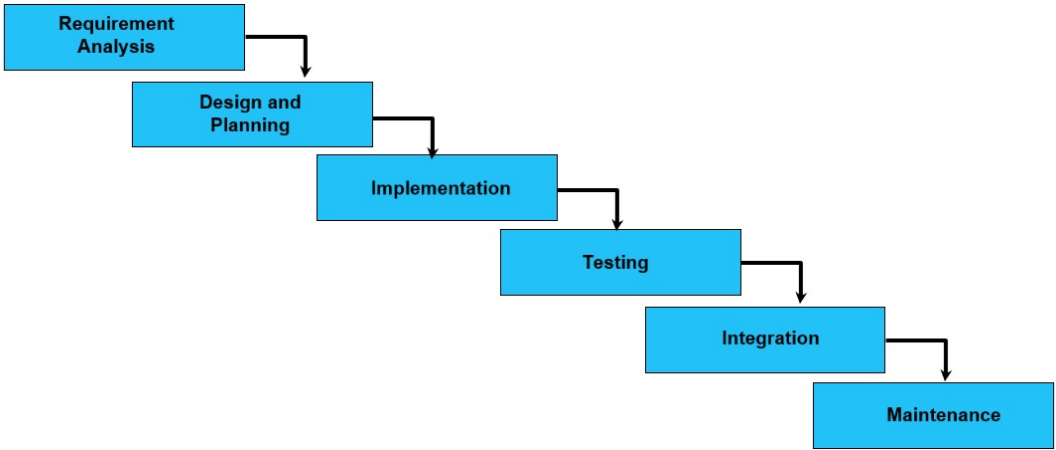
- **No Co**
  - Ai … ng pre-built co…
  - Ex…
  - W…
    - *Sovereignty ?*

Limitations

- **Scalability Issues:** Often unsuitable for high-performance or large-scale systems.
- **Limited Custom Logic:** Difficult to implement complex or domain-specific functionalities.
- **Vendor Lock-in and Sovereignty:** Dependence on the platform's ecosystem and data policies.

Waterfall Process Model

Waterfall Model



Software Process Models: Traditional Methodologies

- **Waterfall Model :**
  - Move from one phase to the next only when its preceding phase is completed and perfected
  - The output from each stage is fed into the next stage in sequence.
  - US department of defence projects attempted to entrench this model by requiring their contractors to produce the waterfall deliverables and then to formally accept them to a certain schedule

    (US military standard DoD-2167)

Features of Waterfall Model

1. **Sequential Approach**: Each phase completed before moving to the next.
2. **Document-Driven:** Heavy reliance on documentation for project definition and goals.
3. **Quality Control:** High emphasis on testing at each phase.
4. **Rigorous Planning**: Careful definition and monitoring of scope, timelines, and deliverables.

Software Process Models: Traditional Methodologies

- **Waterfall Model :**
  - Strength :
    - Works for well-understood problems
    - Forces the software team to complete requirements before design, design before code...
    - It is easy to manage due to the **rigidity** of the model – each phase has specific deliverables and a review process.
    - It can protect the company from the risk of diving into never ending requirements and changes.

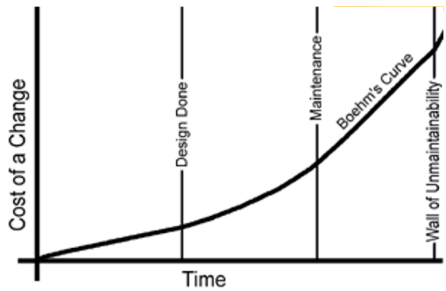
Software Process Models: Traditional Methodologies

Waterfall Model :

- Weakness :
  - Does not allow for changing requirements
  - Does not work for novel or poorly understood problems
  - Lack of user involvement once specification is written
  - Unrealistic separation of specification from design
    - *There are debates on the possibility to get back from design to specification*
  - Doesn't easily accommodate prototyping, reuse ..
  - No working software is produced until late during the life cycle.

Software Process Models: Traditional Methodologies

Waterfall Model:



Barry Boehm's Cost of Change (1981):

Software Engineering Economics

Data from waterfall projects showed that the cost of fixing a defect increases exponentially the later it is found in the development cycle.

Software Process Models: Traditional Methodologies

Waterfall Model: When to use it:

- **Embedded systems:** Because of the inflexibility of hardware, The waterfall model is the most appropriate
- **Critical systems:** Where safety and security are paramount
- **Large projects:** Where clear documentation is necessary
- **Stable Requirements:** No expected changes once phases are completed.

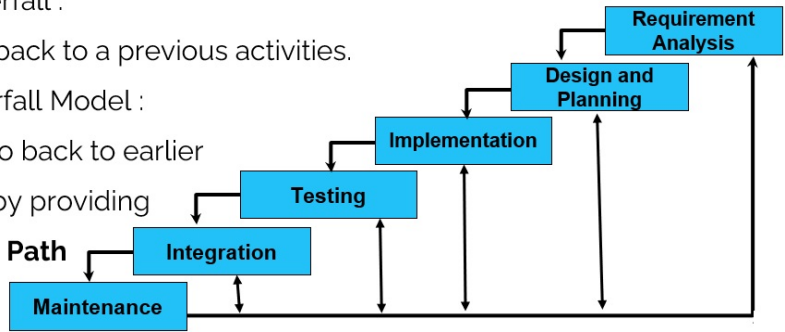
Software Process Models: Traditional Methodologies

Waterfall Model :

- When not to use it :
  - The waterfall model is not the right process model in situations where :
    - Requirements are not well understood
    - Requirements are likely to change
    - The project is novel or innovative
    - Informal communication among teams/members/customer
- Iterative development and agile methods are better for these systems

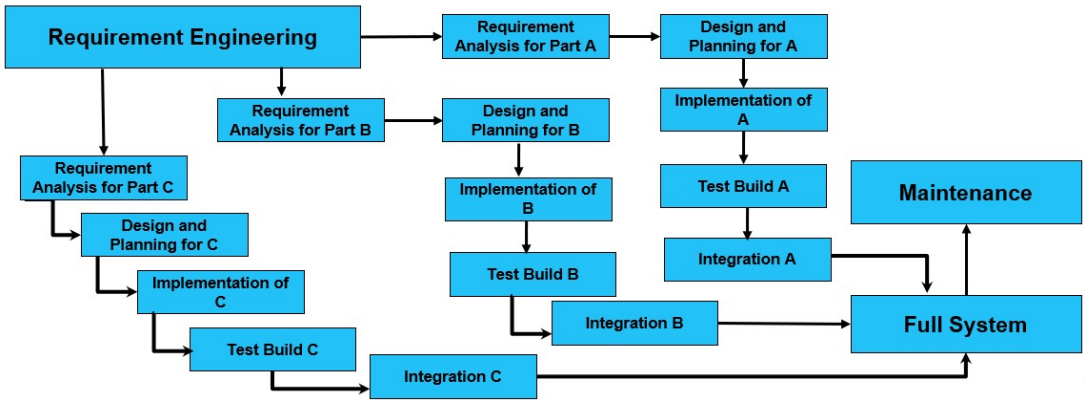
Software Process Models: Traditional Methodologies

- **Waterfall Model :**
  - Classical Waterfall :
    - No going back to a previous activities.
  - Iterative Waterfall Model :
    - You can go back to earlier Activities by providing **Feedback Path**



Software Process Models: Traditional Methodologies

Incremental Process Model:



Software Process Models: Traditional Methodologies

Incremental Process Model:

- Requirements of the Full Software are first broken down into several modules/Subsystems/Set of Functionalities that can be incrementally constructed and delivered consecutively
- Initially : The Development Team ensures the development of core features of the system as the first release.
- Once the core features are fully developed, then new modules/functionalities are added in successive versions.
- Each incremental version is usually developed using an **iterative waterfall** model of development.

Software Process Models: Traditional Methodologies

Incremental Process Model:

- Strength
  - Uses divide and conquer for a breakdown of tasks and complexity reduction.
  - Clients have a clear idea of the project with constant feedback from the client at each release
  - Incremental Resource Deployment.
- Weakness
  - Because of its continuous iterations the cost increases.
  - Lack of an overall vision for the long term plan
  - Potential risk to break a production system

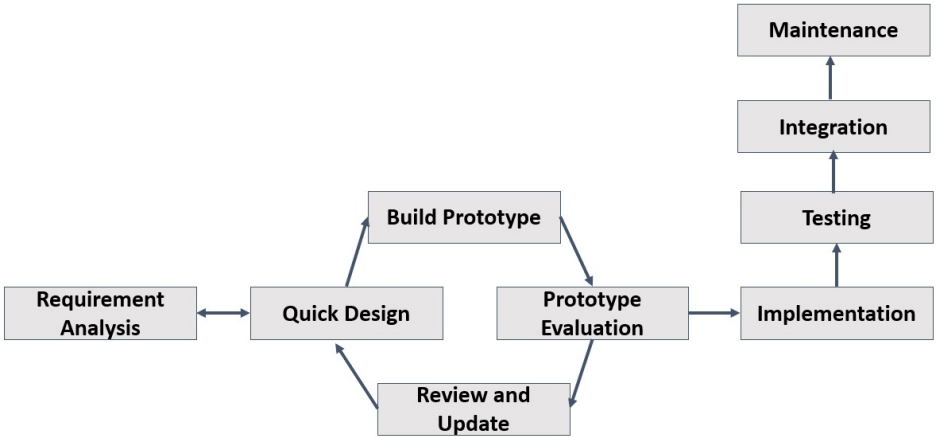
Software Process Models: Traditional Methodologies

Prototyping Process Model:

- Prototyping is the process to produce a replica of the software product with minimal efforts and quickly.
- The prototype should exhibit the main functionality to be engineered. Though, it can be a non-functional system without any business logic
- The aim is to quickly create or build a prototype to clarify the requirements.
- The clients can therefore express clearly their needs by having something similar to the product they are expecting.

Software Process Models: Traditional Methodologies

Prototype Process Model



Software Process Models: Traditional Methodologies

Prototyping Process Model:

- Prototyping is an iterative process requiring designing, building, feedback, refining.
- Once the requirements are fully understood, the customer is happy with the prototype
  - The prototype is disregarded as it does not meet the quality standards
  - The full system is built and implemented based on the prototype

Software Process Models: Traditional Methodologies

Prototyping Process Model:

- Prototyping Tools:
  - Mockups (Pen and Papers, or even software tools)
  - Online tools:
    - Figma, Balsamiq, proto.io „,
  - Visual Tools (Widgets with Drag and Drop) for rapid application development.

Software Process Models: Traditional Methodologies

Prototyping Process Model:

- Strength
  - Better way to get the requirements correctly
- Weakness
  - Can be costly to build *throwaway* prototypes + time consuming
  - Bad Perception/Appreciation about the effort/pricing of the final product:
    - Too fast to be build the prototype $\rightarrow$ to build the software product, must be quick too.

Software Process Models: Traditional Methodologies

Spiral Process Model:

- Initially proposed by Boehm in 1986.
- Spiral Model is a risk-driven software development process model.
- It is a combination of Waterfall model, Iterative model & Prototyping model.
- Software is developed in a series of incremental releases as per each spiral.

Software Process Models: Traditional Methodologies

Spiral Process Model:

- is a risk-driven model where the focus is on managing risk through multiple iterations of the software development process.
- There are many iteration where each iteration of the spiral has **four activities** which represents a complete software development cycle, from requirements gathering and analysis to design, implementation, testing, and maintenance.

What is "Risk" in Software Development?

Risk is any potential problem or uncertainty that could negatively impact the success of a software project.

What is "Risk" in Software Development?

Risk is any potential problem or uncertainty that could negatively impact the success of a software project.

Types of Risks

- **Technical:** Wrong technology choice, performance issues, integration problems
- **Schedule:** Delays, missed deadlines, underestimated effort
- **Cost:** Budget overruns, unexpected expenses
- **Resource:** Staff turnover, skill gaps, unavailable resources
- **Business:** Changing requirements, market shifts, competitor actions
- **Security:** Data breaches, vulnerabilities, compliance failures

What is "Risk" in Software Development?

Risk is any potential problem or uncertainty that could negatively impact the success of a software project.

Types of Risks

- **Technical:** Wrong technology choice, performance issues, integration problems
- **Schedule:** Delays, missed deadlines, underestimated effort
- **Cost:** Budget overruns, unexpected expenses
- **Resource:** Staff turnover, skill gaps, unavailable resources
- **Business:** Changing requirements, market shifts, competitor actions
- **Security:** Data breaches, vulnerabilities, compliance failures

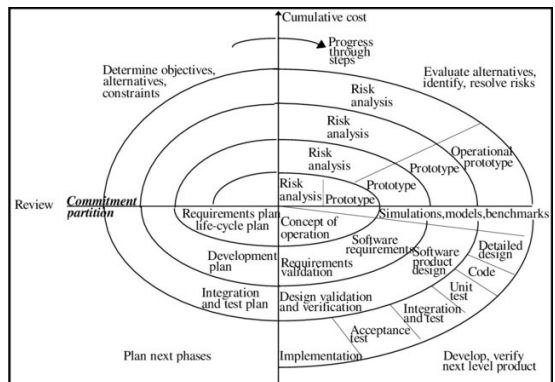
Risk-Driven Approach

The **Spiral Model** explicitly identifies, analyzes, and mitigates risks *before* they become problems — making it well-suited for complex and uncertain projects.

About Spiral Model

Spiral model divided into the four parts:

1. **Planning**
2. **Risk Analysis**
3. **Engineering & Execution**
4. **Evaluate**



Software Process Models: Traditional Methodologies

Spiral Process Model:

- The main activities/four quadrants within each spiral iteration are :
  - **Planning** : Requirements are gathered from the customers and the objectives are identified. Then alternative solutions possible for the phase are proposed in this quadrant. Previous evaluation feedback can be utilized for the planning.
  - **Risk Analysis** : Identify and resolve Risks based on analysing the proposed solutions. At the end of this activity, the Prototype is built for the best possible solution.
  - **Engineering** : The identified features are developed and verified through testing. At the end of the third quadrant, the next version of the software is released to the customer.
  - **Evaluate** : the Customers evaluate the released version of the software.

Advantages and Disadvantages

Advantages:

- Strong focus on risk management
- Flexible and adaptable
- Early user feedback
- Accommodates changes
- Suitable for large projects
- Prototypes reduce risks

Disadvantages:

- Complex to manage
- Requires risk assessment expertise
- Can be costly
- Time-consuming
- Not suitable for small projects
- Success depends on risk analysis

Software Process Models: Traditional Methodologies

Other Process Models for Software Engineering:

- Rapid Application Development
- V Model
- ...

Software Process Models: Traditional Methodologies

Major Drawbacks of Traditional Methodologies

- Lengthy development times:
- Inability to cope with changing requirements
- Too much reliance on heroic developer effort
- Waste/duplication of effort (Example .. Much documentation is required, including documentation that may or may not be needed)

Software Process Models: Modern Methodologies

What's Agile ?

Next Lecture !

Dealines, Team and Project

Next Deadline

09 October 2025: Deadline for Group Project Topics

If you have missing members in your team, please contact your **Lab/Tutorial Instructor** to resolve the issue and ensure teams are balanced.

Team Project Guidelines and Upcoming Phases

Phase 1 – Development Without LLMs (Weeks 1–11)

- At this stage, the use of **Large Language Models (LLMs)** such as ChatGPT, Copilot, etc., is **strictly forbidden**.
- This restriction aims to evaluate your **personal analytical and software development skills**.
- Each team must **design and implement** the project entirely using its own intellectual and technical capacities.
- The final software must be delivered by **Week 11**.

Team Project Guidelines and Upcoming Phases

Phase 2 – LLM-Based development

- After submission, each project will be **swapped** with another team.
- The new team will use a **100% LLM-based approach** to implement the project.
- Both teams will then conduct a **comparison and differentiation analysis** between:
  - Human-only development process
  - LLM-assisted development process